



Fine-Tuning com PEFT: LoRA e QLoRA



O Problema do Fine-Tuning Completo

Fine-Tuning Completo: O Custo

Por que não ajustar todos os parâmetros?

- GPT-3 tem 175 bilhões de parâmetros — cada um precisa de gradiente e otimizadora
- Memória necessária para treinar GPT-3 completo: **~1.2 TB de VRAM (FP32)**
- Inviável em hardware consumidor (RTX 4090 tem 24 GB VRAM)
- Problema adicional: catastrophic forgetting
- **Solução**: treinar apenas um subconjunto pequeno e eficiente de parâmetros

O que é PEFT?

- **Parameter-Efficient Fine-Tuning:** família de métodos que adaptam LLMs sem treinar todos os parâmetros
- **Ideia central:** congelar os pesos pré-treinados e treinar apenas adaptadores
- Principais famílias de métodos PEFT:
 - **Adapter Layers:** insere camadas treináveis entre as camadas do transformer
 - **Prefix Tuning:** otimiza vetores de prefixo no espaço de ativações
 - **LoRA / QLoRA:** decomposição de rank baixo nos pesos de atenção
 - **Prompt Tuning:** aprende tokens de prompt "soft" treináveis

PEFT vs Fine-Tuning Completo

- **Fine-Tuning Completo:**

- Parâmetros treináveis: 100% (ex: 7B para Llama 3)
- VRAM necessária: ~112 GB (FP16) para modelo de 7B
- Risco de overfitting e catastrophic forgetting alto

- **PEFT:**

- Parâmetros treináveis: 0.1% a 1% (ex: ~4M para Llama 3 7B)
- VRAM necessária: ~6–10 GB com QLoRA
- Desempenho comparável ao fine-tuning completo em tarefas específicas

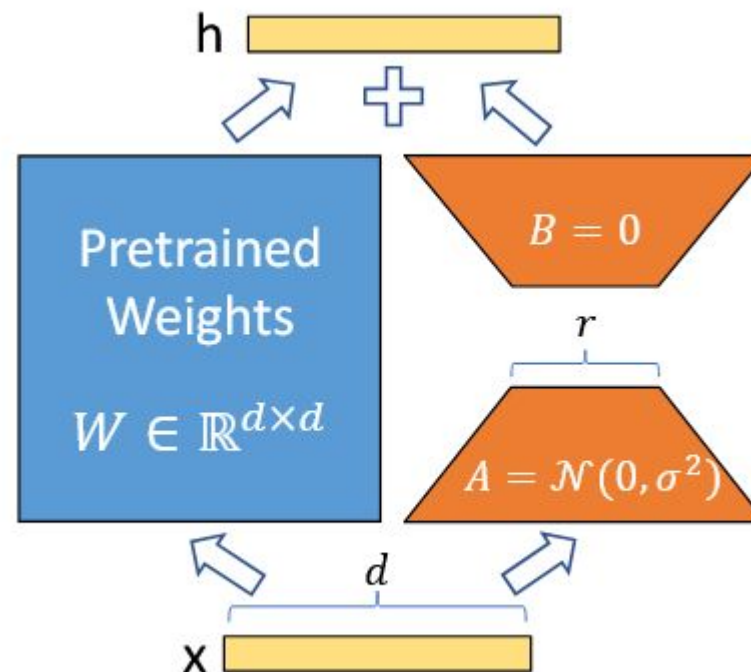


LoRA: Low-Rank Adaptation



LoRA — Low-Rank Adaptation

- Hu et al. (2021) — Microsoft Research



HU, Edward J. et al. **LoRA: Low-Rank Adaptation of Large Language Models**. arXiv, 2021.
Disponível em: <https://arxiv.org/abs/2106.09685>. Acesso em: 15 mar. 2026..

LoRA: Intuição e Fundamento

- Artigo: ["LoRA: Low-Rank Adaptation of Large Language Models" — Hu et al., 2021 \(ICLR 2022\)](#)
- Hipótese central: os updates de pesos durante fine-tuning têm baixa dimensionalidade intrínseca
- Em vez de $W + \Delta W$ (full update), LoRA fatora: $\Delta W = B \times A$
- $A \in \mathbb{R}^{r \times d}$ e $B \in \mathbb{R}^{d \times r}$, onde $r \ll d$ (rank muito menor que dimensão original)
- Durante treino: W congelado, apenas A e B são atualizados
- Durante inferência: $W' = W + BA$ pode ser fundido — zero overhead de latência

LoRA: A Matemática

- Camada linear original: $h = Wx$

- Com LoRA:

$$h = Wx + \Delta Wx = Wx + BAx$$

- **Inicialização:** $A \sim N(0, \sigma^2)$ (gaussiana aleatória), $B = 0$ ($\Delta W = 0$ no início)
- **Escalonamento:** ΔW multiplica por α/r , onde α é um hiperparâmetro (lora_alpha)
- O rank r controla a expressividade: $r=4$ a $r=64$ é o mais comum
- **Resultado:** 10.000x menos parâmetros treináveis em modelos grandes

LoRA: Onde Aplicar no Transformer

- LoRA é aplicado tipicamente nas matrizes de projeção da atenção:
 - W_Q (Query), W_K (Key), W_V (Value), W_O (Output projection)
- Também pode ser aplicado nas camadas MLP (W_{up} , W_{down} , W_{gate})
- No Unsloth com Llama 3.2, os módulos padrão são:
 - `target_modules = ["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]`
- Hu et al. (2021): aplicar em W_Q e W_V já captura grande parte do ganho

LoRA: Resultados (Hu et al., 2021)

- Avaliação em GPT-3 175B, RoBERTa, DeBERTa, GPT-2
- LoRA com $r=4$ **iguala ou supera fine-tuning** completo em GLUE, SuperGLUE e E2E NLG
- 25% de speedup no treinamento por não calcular gradientes dos pesos congelados
- **Sem overhead de latência** em inferência (pesos podem ser fundidos)
- Permite trocar de tarefa rapidamente: swap do adapter A+B sem recarregar modelo base



QLoRA: Quantized Low-Rank Adaptation



O que é Quantização?

- **Reduz a precisão numérica** dos pesos do modelo
- Em vez de float32 (32 bits), usa float16, int8, int4, NF4
- Diminui memória de GPU necessária para inferência
- Reduz largura de banda e latência de execução
- **Objetivo:** menor impacto possível na qualidade

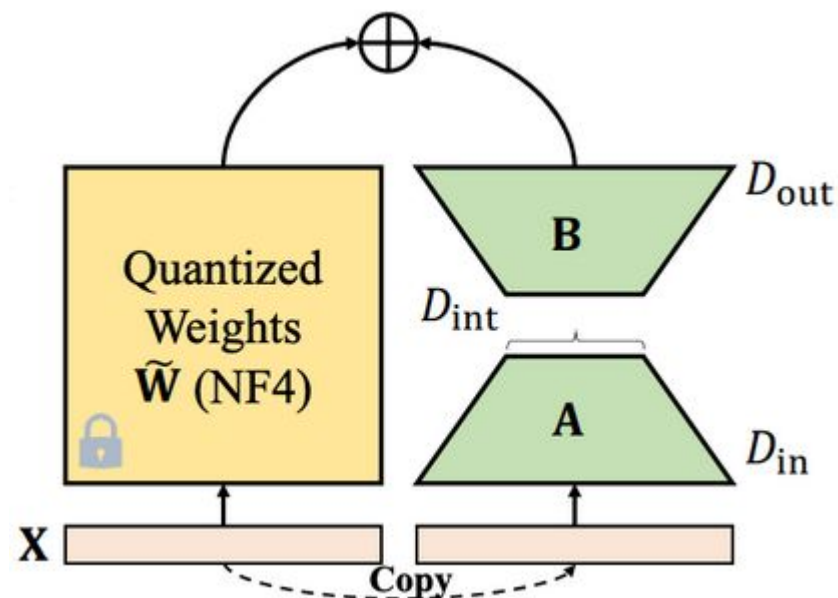
Previsões Numéricas: De float32 a int1



Fonte: Elaboração própria. Redução de memória por parâmetro em cada formato numérico

QLoRA

QLoRA: Intuição e Artigo



XU, Yuhui; XIE, Lingxi; GU, Xiaotao; CHEN, Xin; CHANG, Heng; ZHANG, Hengheng; CHEN, Zhengsu; ZHANG, Xiaopeng; TIAN, Qi.
QA-LoRA: Quantization-Aware Low-Rank Adaptation of Large Language Models. 2023. [Link](#).

QLoRA: Intuição e Artigo

- Artigo: ["QLoRA: Efficient Finetuning of Quantized LLMs" — Dettmers et al., NeurIPS 2023](#)
- Inovação: combina **LoRA** com **quantização** do modelo base em 4-bit (NF4)
- Três contribuições técnicas principais:
 - **4-bit NormalFloat (NF4)**: novo tipo de dado otimizado para pesos com distribuição normal
 - **Double Quantization**: quantiza os constantes de quantização, economizando ~0.37 bits/parâmetro
 - **Paged Optimizers**: usa memória unificada GPU/CPU para evitar OOM em picos de memória

QLoRA: NF4 — NormalFloat 4-bit

- Pesos pré-treinados de LLMs seguem distribuição normal centrada em zero
- NF4 aloca os 16 níveis de quantização de forma ótima para essa distribuição
- Baseado no teorema de quantização de informação: minimiza erro esperado para $N(0,1)$
- Resultado: NF4 é informativamente mais denso que INT4 ou FP4
- O modelo base fica em NF4 (congelado); adaptadores LoRA ficam em BF16/FP16

QLoRA: Redução de Memória

- **Fine-tuning completo** de Llama 65B: ~780 GB VRAM — impossível
- QLoRA com NF4 + Double Quantization + LoRA:
 - Llama 2 7B: de ~112 GB para ~6 GB de VRAM
 - Llama 2 13B: de ~200 GB para ~10 GB de VRAM
 - Llama 2 65B: de ~780 GB para ~48 GB de VRAM (cabe em 1 A100)
- Guanaco 65B treinado em 1 GPU A100 (24h) atingiu 99.3% do desempenho do ChatGPT no Vicuna benchmark (80 perguntas, distribuídas em 9 categorias como escrita, roleplay, matemática, programação e conhecimento geral)

QLoRA: Fluxo de Precisão Mista

- O fluxo de computação no QLoRA usa três tipos de precisão:
 - **4-bit NF4**: pesos do modelo base — armazenados comprimidos, só descomprimidos para computar
 - **BF16**: adaptadores LoRA (A, B) e cálculos de gradiente
 - **FP32**: operadores de normalização para estabilidade numérica
- A descompressão NF4 $\rightarrow$ BF16 **ocorre on-the-fly** durante o forward pass
- Os gradientes fluem apenas pelos adaptadores A e B (os pesos NF4 são congelados)

LoRA vs QLoRA: Comparativo

- **LoRA (Hu et al., 2021):**
 - Modelo base em FP16/BF16 — precisa de mais VRAM
 - Adapters com rank r treináveis nas projeções de atenção (e MLP)
 - Sem overhead de inferência quando fundido
- **QLoRA (Dettmers et al., 2023):**
 - Modelo base em NF4 (4-bit) — ~4x menos VRAM vs FP16
 - Mesma estrutura de adaptadores LoRA, mas com paged optimizers
 - Pequeno overhead de descompressão durante inferência se não fundido
- **Ambos:** desempenho comparável ao fine-tuning completo em tarefas específicas



Hiperparâmetros do LoRA e QLoRA



Hiperparâmetros

- **r (rank):** dimensão das matrizes A e B — controla expressividade
 - r=4: muito leve, tarefas simples; r=16: equilíbrio; r=64: tasks complexas
 - Mais r = mais parâmetros, mais VRAM, mais capacidade de aprendizado
- **lora_alpha (α):** fator de escala do adapter — regra prática: $\alpha = 2r$
 - Escala efetiva: α/r — $\alpha=16$ com $r=8$ dá escala 2.0
- **lora_dropout:** regularização nos adaptadores — 0.0 a 0.1 é comum
- **bias:** "none" é padrão (não treinar bias); "lora_only" ou "all" são alternativas

Hiperparâmetros

- **target_modules:** quais camadas recebem adapters LoRA
 - Llama 3.2 recomendado: q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
 - Incluir MLP (gate/up/down) geralmente melhora resultados em tarefas generativas
- **task_type:** tipo de tarefa para a cabeça do modelo
 - CAUSAL_LM: geração de texto autoregressivo (Llama, GPT)
 - SEQ_2_SEQ_LM: encoder-decoder (T5, BART)
 - SEQ_CLS: classificação de sequência
- **use_rslora:** Rank-Stabilized LoRA (Kalajdzievski, 2023) — normaliza escala por $\sqrt{r}$



Fine-Tuning: Formato dos Dados



Formatos de Dados: Visão Geral

- Dois paradigmas principais para fine-tuning instrucional:
 - **Instruction Tuning (Alpaca style):** pares instrução → resposta
 - **Chat / Conversational (ShareGPT style):** multi-turn com roles
- O Unsloth **suporta ambos** os formatos nativamente via *standardize_sharegpt()*
- O formato determina como o template de chat é aplicado ao texto de entrada
- Llama 3.2 usa o template de chat oficial da Meta com tokens especiais

Formato Alpaca (Instruction)

- Estrutura JSON com campos: instruction, input (opcional), output
- Exemplo de registro:

```
{"instruction": "Resuma o texto a seguir em 3 pontos.",  
  "input": "O Brasil possui a maior..."),  
  "output": "1. O Brasil ... 2. ... 3. ..."}}
```

- Quando input está vazio, o prompt simplifica para:

```
"### Instrução:\n{instruction}\n### Resposta:\n{output}"
```

- Bom para: tarefas de instrução simples, classificação, resumo, extração

Formato ShareGPT (Conversacional)

- Estrutura JSON com campo "conversations": lista de turnos com "from" e "value"
- Roles: "system" (opcional), "human" (usuário), "gpt" (modelo)
- Exemplo:

```
{"conversations": [  
  {"from": "system", "value": "Você é um assistente..."},  
  {"from": "human", "value": "Como funciona o LoRA?"},  
  {"from": "gpt", "value": "LoRA funciona..."}]}
```

- Bom para: chatbots, assistentes multi-turn, roleplay, RAG conversacional

Template de Chat do Llama 3.2

- O Llama 3.2 usa o formato ChatML da Meta com tokens especiais:

`<|begin_of_text|>`

`<|start_header_id|>system<|end_header_id|>`

`Você é um assistente útil.<|eot_id|>`

`<|start_header_id|>user<|end_header_id|>`

`Explique LoRA.<|eot_id|>`

`<|start_header_id|>assistant<|end_header_id|>`

`LoRA é...<|eot_id|><|end_of_text|>`

- O Unsloth aplica esse template automaticamente via `get_chat_template("llama-3.2")`



Fine-Tuning: Pipeline Completa



Unsloth: Instalação e Setup

- Unsloth otimiza o treinamento com kernels CUDA customizados (2x mais rápido, 70% menos VRAM)
- Instalação (Colab/Linux com CUDA):

```
pip install unsloth
```

```
# Ou para versão mais recente:
```

```
pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
```

- Suporte oficial: NVIDIA GPUs (Ampere+, Turing), bfloat16, Flash Attention 2
- Modelos suportados: Llama 3.x, Mistral, Gemma, Phi-3, Qwen, TinyLlama e outros

Carregamento do Modelo Base

- Carregando Llama 3.2 com QLoRA (4-bit):

```
from unsloth import FastLanguageModel
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Llama-3.2-3B-Instruct",
    max_seq_length = 2048,      # contexto máximo
    dtype = None,              # auto: BF16 em Ampere+
    load_in_4bit = True,       # QLoRA: NF4 quantization
)
```

- max_seq_length controla o comprimento das sequências de treinamento
- load_in_4bit=True ativa QLoRA; False usa LoRA com modelo em BF16

Configurando o Adapter LoRA

- Adicionando adapters LoRA ao modelo carregado:

```
model = FastLanguageModel.get_peft_model(  
    model,  
    r = 16,                # rank do adapter  
    lora_alpha = 16,        # escala (alpha=r é comum)  
    lora_dropout = 0.0,     # 0 funciona bem com Unsloth  
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",  
                      "gate_proj", "up_proj", "down_proj"],  
    bias = "none",  
    use_gradient_checkpointing = "unsloth", # economiza 30% VRAM  
    use_rslora = False,  
)
```

Preparando o Dataset (Alpaca)

- Carregando e formatando o dataset no formato Alpaca:

```
from datasets import load_dataset
from unsloth.chat_templates import get_chat_template, standardize_sharegpt

tokenizer = get_chat_template(tokenizer, chat_template="llama-3.2")
dataset = load_dataset("tatsu-lab/alpaca", split="train")

def format_prompts(examples):
    return {"text": [alpaca_prompt.format(
        examples["instruction"][i], examples["input"][i],
        examples["output"][i]) for i in range(len(examples["instruction"]))]}
dataset = dataset.map(format_prompts, batched=True)
```

Preparando o Dataset (Chat)

- Para datasets conversacionais no formato ShareGPT:

```
dataset = load_dataset("mlabonne/FineTome-100k", split="train")
# Padroniza roles: human/gpt -> user/assistant
dataset = standardize_sharegpt(dataset)

def formatting_prompts_func(examples):
    convos = examples["conversations"]
    texts = [tokenizer.apply_chat_template(
        c, tokenize=False, add_generation_prompt=False)
        for c in convos]
    return {"text": texts}
dataset = dataset.map(formatting_prompts_func, batched=True)
```

Treinamento com SFTTrainer

- O Unsloth integra com o TRL SFTTrainer:

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = dataset,
    dataset_text_field = "text",
    max_seq_length = 2048,
    dataset_num_proc = 2,
    packing = False, # True empacota sequências curtas
    args = TrainingArguments(...),
)
```

TrainingArguments: Config. Recomendada

- Configuração recomendada para Llama 3.2 3B com QLoRA:

```
TrainingArguments(  
    per_device_train_batch_size = 2,  
    gradient_accumulation_steps = 4, # batch efetivo = 8  
    warmup_steps = 5,  
    num_train_epochs = 3,           # ou max_steps=60 p/ teste  
    learning_rate = 2e-4,  
    fp16 = not bf16_supported,     # BF16 em Ampere+  
    bf16 = bf16_supported,  
    logging_steps = 1,  
    optim = "adamw_8bit",          # economiza VRAM  
    lr_scheduler_type = "cosine",  
    output_dir = "outputs",  
)
```

Salvando o Modelo Treinado

- Após treino, salvar o adapter LoRA (leve, ~100MB) ou o modelo fundido:
- Salvar apenas o adapter (recomendado para iteração rápida):

```
model.save_pretrained("meu_modelo_lora")  
tokenizer.save_pretrained("meu_modelo_lora")
```
- Salvar o modelo fundido em FP16 (para deploy):

```
model.save_pretrained_merged("meu_modelo_merged", tokenizer, save_method="merged_16bit")
```
- Salvar em GGUF para inferência local com llama.cpp / Ollama:

```
model.save_pretrained_gguf("meu_modelo_gguf", tokenizer, quantization_method="q4_k_m")
```

Inferência Após Fine-Tuning

- Para inferência com o modelo fine-tuned:

```
FastLanguageModel.for_inference(model) # ativa kernels 2x mais rápidos

messages = [{"role": "user", "content": "Explique LoRA em 3 pontos."}]

inputs = tokenizer.apply_chat_template(
    messages, tokenize=True, add_generation_prompt=True,
    return_tensors="pt").to("cuda")

outputs = model.generate(input_ids=inputs,
    max_new_tokens=512, temperature=0.7,
    do_sample=True, pad_token_id=tokenizer.eos_token_id)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```



Fine-Tuning: Boas Práticas



Boas Práticas: Qualidade dos Dados

- **Qualidade > Quantidade:** 1.000 exemplos de alta qualidade superam 100k medíocres
- **Diversidade de formatos:** misturar instrução simples + raciocínio + chain-of-thought
- **Evitar:** repetição excessiva, outputs truncados, exemplos ambíguos ou contraditórios
- **Validar com amostragem:** revisar 10% do dataset manualmente antes de treinar
- Incluir exemplos do **domínio-alvo + exemplos gerais** para evitar catastrophic forgetting

Boas Práticas: Treinamento

- **Monitorar a loss de treinamento** — deve cair consistentemente, sem explosão
- Train loss ~1.0 a 1.5 em tarefas de instrução é razoável
- Usar **packing=True** se os exemplos forem curtos (melhora throughput)
- **gradient_accumulation_steps**: simula batch maior sem aumentar VRAM
- **Early stopping**: avaliar em validation set a cada N steps para não overfit
- **LR Scheduler**: cosine ou linear com warmup de 5–10% dos steps totais
- **adamw_8bit (bitsandbytes)**: reduz VRAM do otimizador em ~75% vs AdamW FP32

Boas Práticas: Escolha de r e α

- **Regra geral:** começar com $r=16$, $\text{lora_alpha}=16$ e ajustar conforme o caso
- **Para tarefas simples (classificação, extração):** $r=4$ ou $r=8$ é suficiente
- **Para geração criativa/raciocínio complexo:** $r=32$ ou $r=64$
- **$\text{lora_alpha} = r$:** mantém escala efetiva em 1.0 (neutro)
- **$\text{lora_alpha} = 2r$:** aumenta a contribuição do adapter — útil quando dataset é pequeno
- Aglawe et al. (2023) "LoRA Land": $r=16$ com $\alpha=32$ atingiu melhor avg em 25 tarefas
- Rank muito alto ($r>128$) raramente melhora — aumenta overfitting e VRAM



PUC Minas

© **PUC Minas** • Todos os direitos reservados, de acordo com o art. 184 do Código Penal e com a lei 9.610 de 19 de fevereiro de 1998.

Proibidas a reprodução, a distribuição, a difusão, a execução pública, a locação e quaisquer outras modalidades de utilização sem a devida autorização da Pontifícia Universidade Católica de Minas Gerais.